Guion de video tecnico

Plataforma MLOps para prediccion de readmision temprana en diabetes

Duracion objetivo 9 a 10 minutos	**Formato** 1, 2 o 3 participantes
Meta Explicar arquitectura, ejecucion y evidencia	**Regla** Mostrar evidencia, no solo diapositivas

Como usar este guion

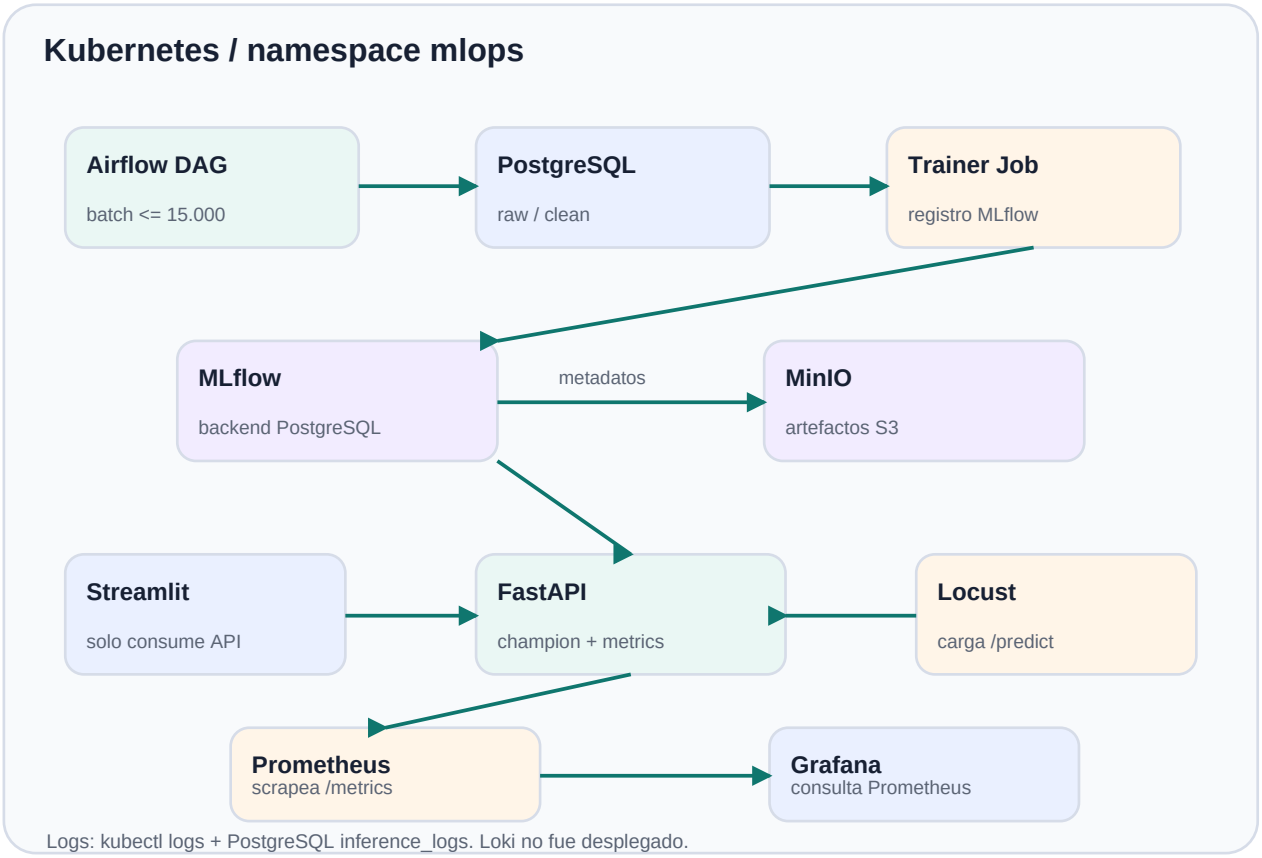
No hay roles fijos. Cada bloque puede leerlo cualquier integrante disponible. Si presentan tres personas, alternen por bloques; si presentan dos, repartan bloques consecutivos; si presenta una sola persona, lea todas las locuciones y mantenga las transiciones.

Antes de grabar: abra las pestañas de Kubernetes, Airflow, MLflow, Streamlit, Grafana y Locust. Mantenga disponibles las capturas de la arquitectura, el DAG en verde, la prediccion y Grafana con carga. No revele contrasenas ni secretos.

Las frases entre comillas son una guia natural, no un texto que deba recitarse de memoria. Conserve los numeros, nombres de recursos y hechos tecnicos. Si el tiempo se aprieta, omitan una frase de contexto, nunca la evidencia de pantalla.

Arquitectura tecnica: conexiones reales

EN PANTALLA: Usar esta pagina como la primera imagen completa del video; recorrer las flechas de izquierda a derecha y de arriba abajo.



Punto clave para la sustentacion: Grafana no consume la API directamente. Grafana consulta Prometheus, y Prometheus scrapea el endpoint /metrics de FastAPI. Tampoco se debe afirmar que existe Loki: en esta implementacion los logs se inspeccionan con kubectl logs y la auditoria estructurada vive en PostgreSQL.

00:00 - 00:45 - Apertura: problema y objetivo

EN PANTALLA: Titulo del proyecto y diagrama de arquitectura actualizado.

LOCUCION: En este proyecto construimos una plataforma MLOps para estimar el riesgo de readmision hospitalaria temprana, antes de 30 dias, en pacientes con diabetes.

LOCUCION: No buscamos diagnosticar diabetes. El dataset contiene encuentros clinicos de pacientes con diabetes y la variable objetivo identifica si existe readmision en menos de 30 dias.

LOCUCION: El reto no era solo entrenar un modelo: debiamos automatizar datos, versionar el modelo, servir predicciones, auditarlas y observar el sistema bajo carga.

00:45 - 02:00 - Kubernetes: por que y como se integra

EN PANTALLA: Diagrama tecnico anterior; luego terminal con nodo Ready y kubectl top nodes.

LOCUCION: Kubernetes fue la capa de orquestacion. Docker construye las imagenes; Kubernetes las ejecuta como Pods y mantiene los servicios disponibles.

LOCUCION: Usamos Deployments para aplicaciones sin estado, como FastAPI, Streamlit, MLflow, Airflow, Prometheus, Grafana y Locust. Para PostgreSQL y MinIO usamos StatefulSets con almacenamiento persistente.

LOCUCION: Los Services dan nombres internos estables. Por eso Streamlit llama a diabetes-api y no depende de una direccion IP de pod que pueda cambiar.

LOCUCION: Los manifiestos YAML son la receta reproducible: describen imagen, configuracion, recursos, probes, servicios y volumenes. Con kubectl apply declaramos el estado que debe mantener el cluster.

LOCUCION: En la VM usamos K3s. Un detalle importante fue evitar el solapamiento entre la red fisica de la VM y el rango de Services de Kubernetes: configuramos Pods en 10.244.0.0/16 y Services en 10.245.0.0/16. Asi metrics-server pudo alcanzar el kubelet y kubectl top nodes quedo funcional.

LOCUCION: Hacia el exterior no usamos Ingress. Los Services permanecen ClusterIP y exponemos temporalmente las interfaces por kubectl port-forward dentro del rango de puertos permitido por la VM. Es una decision adecuada para una entrega local y reduce superficie de exposicion.

02:00 - 02:55 - Retos reales de infraestructura

EN PANTALLA: Terminal con DiskPressure resuelto, imagenes publicadas en Docker Hub y pods actuales saludables.

LOCUCION: El primer reto fue DiskPressure. Rancher embebido, imagenes antiguas y capas de contenedores ocupaban el disco de la VM, causando expulsiones de pods. La solucion no fue borrar a ciegas: identificamos las imagenes y contenedores de practicas antiguas, los retiramos de forma dirigida y verificamos que DiskPressure quedara en False.

LOCUCION: El segundo reto fue la reproducibilidad de imagenes. Las aplicaciones propias - MLflow, entrenador, API, Airflow y Streamlit - se construyeron, versionaron y publicaron en Docker Hub. Kubernetes las consume por digest cuando corresponde, para evitar que el mismo tag cambie silenciosamente.

LOCUCION: Tambien resolvimos incompatibilidades reales: MLflow 2.17.2 fallo con SQLAlchemy 2.1, por lo que fijamos SQLAlchemy 2.0.54. MinIO requirio una imagen mantenida compatible con el entorno. Estos ajustes son parte de operar MLOps, no pasos decorativos.

02:55 - 04:10 - Datos, trazabilidad y entrenamiento

EN PANTALLA: PostgreSQL, MLflow y MinIO; tabla o consulta de raw y clean.

LOCUCION: La persistencia se separo en tres capas: raw para datos originales, clean para caracteristicas procesadas y inference_logs para auditar cada prediccion del servicio.

LOCUCION: El dataset tiene 101.766 encuentros. La clase positiva, readmision menor a 30 dias, representa 11,16 por ciento, por lo que usamos metricas de la clase positiva y no solo accuracy.

LOCUCION: MLflow guarda experimentos, parametros, metricas, versiones y el registro del modelo. PostgreSQL funciona como backend de metadatos y MinIO conserva los artefactos mediante almacenamiento compatible con S3.

LOCUCION: El primer entrenamiento registro el modelo diabetes_readmission, version 1, y lo promovio al alias champion. Sus metricas de prueba fueron F1 positiva de 0,2625, recall de 0,55 y ROC-AUC de 0,6516.

LOCUCION: La seleccion no se basa en accuracy porque la clase de readmision temprana solo representa 11,16 por ciento. Usamos F1 de la clase positiva para equilibrar precision y recall, y dejamos ROC-AUC y PR-AUC como metricas complementarias.

04:10 - 05:25 - Airflow: pipeline reproducible

EN PANTALLA: Airflow con diabetes_mlops_pipeline y todas las tareas verdes; luego consulta del batch 15.000.

LOCUCION: Airflow transforma el proceso en un DAG, es decir, un flujo ordenado de tareas con dependencias. Asi el pipeline se puede repetir, monitorear y recuperar si una etapa falla.

LOCUCION: El DAG valida la fuente, carga datos incrementalmente en lotes maximos de 15.000 registros, ejecuta controles de calidad, transforma, divide datos, entrena, registra en MLflow, compara y promueve el champion.

LOCUCION: Como evidencia, ejecutamos el lote airflow-batch-001. La consulta en PostgreSQL muestra 15.000 registros en raw y 15.000 en clean. Esto demuestra la carga incremental y la trazabilidad entre capas.

LOCUCION: Encontramos un problema operativo adicional: el liveness probe original del scheduler ejecutaba una comprobacion que superaba su timeout y provocaba reinicios. Lo sustituimos por una verificacion ligera de que el proceso airflow scheduler esta vivo y validamos un pod nuevo sin reinicios. La leccion es que un probe debe medir salud, no introducir una carga que cause falsos reinicios.

05:25 - 06:40 - Serving: FastAPI y Streamlit

EN PANTALLA: Streamlit, una prediccion completada y la fila correspondiente en inference_logs.

LOCUCION: FastAPI es el unico servicio que sirve el modelo. Expone health, model-info, predict y metrics. Al iniciar o al recibir una prediccion, carga dinamicamente el modelo que tenga el alias champion en MLflow.

LOCUCION: Esto evita dejar una version fija del modelo dentro del codigo de la API. Si se promueve una nueva version champion, el serving puede consumirla sin cambiar el contrato de la aplicacion.

LOCUCION: Streamlit es la interfaz de usuario y consume exclusivamente FastAPI. No consulta directamente PostgreSQL, MinIO ni MLflow. Esa separacion simplifica seguridad, trazabilidad y evolucion del sistema.

LOCUCION: Cada prediccion queda guardada con request_id, entrada, salida, score, version de modelo y latencia. Por eso podemos explicar no solo que hubo una respuesta, sino que existe auditoria de produccion.

LOCUCION: La API valida el esquema de caracteristicas antes de inferir. Probamos intencionalmente una peticion incompleta y recibimos 422; ese evento incrementa diabetes_prediction_errors_total con razon validation. Es una prueba de contrato y de observabilidad, no un error accidental.

06:40 - 07:50 - Observabilidad: Prometheus y Grafana

EN PANTALLA: Grafana funcionando: solicitudes exitosas, error controlado, RPS y latencia p95.

LOCUCION: FastAPI publica metricas en el endpoint metrics. Prometheus lo consulta periodicamente y verificamos que el target diabetes-api estuviera up igual a uno.

LOCUCION: Grafana convierte esas series en evidencia operativa: cantidad de predicciones exitosas, errores de validacion, predicciones por segundo y latencia p95.

LOCUCION: El error de validacion visible fue provocado de forma controlada con una solicitud incompleta. No es una falla oculta: demuestra que la API valida el contrato y que el sistema cuenta los errores.

LOCUCION: Durante la prueba, el dashboard registro 1.868 predicciones exitosas, 26,3 solicitudes por segundo y una latencia p95 de 204,23 milisegundos.

LOCUCION: La latencia p95 se calcula desde el histograma publicado por la API: no es un promedio que esconda casos lentos. Por eso sirve para detectar degradacion de la experiencia bajo carga.

LOCUCION: Aclaracion tecnica: Grafana es la capa de visualizacion y Prometheus es la capa de recoleccion y consulta de metricas. Loki no forma parte de este despliegue; la evidencia de logs viene de kubectl logs y de inference_logs en PostgreSQL.

07:50 - 08:45 - Prueba de carga: Locust

EN PANTALLA: Locust mostrando /predict, usuarios, RPS y cero fallos; alternar brevemente con Grafana.

LOCUCION: Locust simula usuarios concurrentes y envio solicitudes validas al endpoint predict dentro de la red del cluster. Asi probamos el servicio en una condicion mas realista que una sola llamada manual.

LOCUCION: Se ejecuto una carga moderada con 10 usuarios y una tasa de arranque de 2 usuarios por segundo. Mientras Locust generaba trafico, Grafana se actualizo en tiempo real.

LOCUCION: La conexion que demostramos es Locust hacia FastAPI, FastAPI hacia el modelo champion, registro en PostgreSQL y metricas hacia Prometheus y Grafana. No son pruebas separadas; son una misma cadena de evidencia.

LOCUCION: El script de Locust vive en un ConfigMap y usa un payload valido de 44 caracteristicas. Esto evita resultados falsos: si la carga enviara datos incompletos, solo estariamos midiendo errores de validacion y no la capacidad real de inferencia.

08:45 - 09:40 - Cierre: resultado y siguientes pasos

EN PANTALLA: Consulta final de PostgreSQL y mapa de arquitectura con todos los componentes tecnicos en verde.

LOCUCION: Al cierre quedaron 7.884 inferencias auditadas en PostgreSQL, con latencia promedio de 57,21 milisegundos. Esto confirma que el trafico de carga llego a la auditoria de produccion.

LOCUCION: La plataforma integra datos, entrenamiento, versionamiento, orquestacion, serving, interfaz, monitoreo y prueba de carga sobre Kubernetes. Las imagenes propias se publicaron en Docker Hub y los servicios se declaran mediante manifiestos reproducibles.

LOCUCION: Como mejoras futuras, podemos experimentar con otros modelos, optimizar la metrica clinica, incorporar alertas de Grafana y ejecutar nuevas versiones del DAG. Pero la arquitectura MLOps solicitada ya quedo funcional y evidenciada de extremo a extremo.

LOCUCION: Gracias. Estamos listos para responder preguntas sobre cualquier componente o sobre las decisiones de arquitectura.

Checklist de grabacion

Momento	Que mostrar
Antes	Nodo Ready; pods/servicios/PVC sanos; port-forwards activos; no mostrar secretos.
Evidencia 1	Diagrama de arquitectura y breve explicacion de Kubernetes.
Evidencia 2	Airflow con DAG verde y consulta del batch incremental de 15.000.
Evidencia 3	MLflow con champion v1 y MinIO como almacenamiento de artefactos.
Evidencia 4	Streamlit predice por FastAPI; PostgreSQL muestra inference_logs.
Evidencia 5	Locust genera carga; Grafana muestra RPS, p95, exitosas y error controlado.
Cierre	Consulta de 7.884 inferencias auditadas y latencia promedio 57,21 ms.

Consejo final: hablen de decisiones y evidencia. En lugar de decir "montamos Grafana", muestren que Grafana observó 1.868 predicciones, 26,3 RPS y p95 de 204,23 ms durante Locust.

Preguntas que podria hacer el profesor: Por que no SQLite? Porque MLflow usa PostgreSQL como backend persistente y MinIO para artefactos. Por que no Ingress? Porque el alcance es una VM local con puertos restringidos; ClusterIP mas port-forward reduce complejidad. Donde estan los logs? En kubectl logs y en PostgreSQL; Loki no se despliego. Que asegura reproducibilidad? Manifiestos, imagenes versionadas en Docker Hub y dependencias fijadas.